LingTu 导航规划栈 — 从全局到局部

LingTu 导航规划栈 — 从全局到局部
- 0. 一屏速览
- 1. PCT_Planner 是什么,现状
- 2. Tomogram — PCT 的输入
- 3. 全局规划 — `_PCTBackend.plan()` 展开
- 4. A* 备胎 — `_AStarBackend`
- 5. 全局 → 局部 解耦:WaypointTracker
- 6. 局部规划 — CMU 预采样路径
- 7. 路径跟踪 — PathFollower (Pure Pursuit)
- 8. CmdVelMux — 优先级仲裁
- 9. NavigationModule 状态机
- 10. 和语义层的关系
- 11. 关键文件快查
- 12. 已知问题 / 技术债
- 13. 性能汇总(S100P aarch64)
- 14. 想深入看哪块
- 15. 相关文档

LingTu 导航规划栈 — 从全局到局部

> 对象:想理解 LingTu 是怎么把一个 `(x, y)` 目标变成机器狗动作的工程师。 假设:你懂 ROS2 / 点云 / A* 基础。 最后更新:2026-04-19

---

0. 一屏速览

```
用户点目标 (x, y)
    ↓
GatewayModule.goal_pose
    ↓
NavigationModule._plan()          ← 全局规划入口
    ↓
GlobalPlannerService              ← 选 PCT / A*
    ↓
_PCTBackend.plan()                ← C++ ele_planner.so + traj_opt.so
    ↓
global_path (N, 3)                ← 含 z,处理台阶
```

```
    ↓
NavigationModule 每帧发 waypoint    ← WaypointTracker 推进
    ↓
LocalPlannerModule                  ← CMU 预采样 + 打分避障
    ↓
local_path (20-30 点)
    ↓
PathFollowerModule                  ← Pure Pursuit
    ↓
CmdVelMux (优先级仲裁)
    ↓
ThunderDriver / ROS2SimDriver → motors
```

---

1. PCT_Planner 是什么,现状

来源与许可

- **出处**:HKU/HKUST,Bowen Yang & Jie Cheng 开源(GPLv2)
- **路径**: `src/global_planning/PCT_planner/`
- **论文**:[Efficient Trajectory Planning for Autonomous Off-Road Driving in Unstructured Environments]
- 我们**直接集成,没 fork**,保留上游结构便于跟进

目录结构

```
src/global_planning/PCT_planner/
├── planner/
│   ├── lib/                        ← 编译产物 (aarch64)
│   │   ├── ele_planner.so          ← 3D A*(核心)
│   │   ├── a_star.so
│   │   ├── traj_opt.so             ← GPMP 轨迹优化
│   │   └── libele_planner_lib.so
│   ├── scripts/
│   │   └── planner_wrapper.py      ← TomogramPlanner Python 包装
│   └── src/ele_planner/            ← C++ 源码
├── tomography/                     ← tomogram 构建算法
├── launch/                         ← ROS2 launch(未使用)
└── rsc/                            ← 示例 pcd / tomogram
```

集成层

`src/global_planning/pct_adapters/src/global_planner_module.py`: - 注册 `@register("planner_backend", "pct")` → `_PCTBackend` - 启动时尝试 `import planner_wrapper.TomogramPlanner`,加载 tomogram.pickle - 失败(.so 不在,x86)时**不 crash**,降级到 `_AStarBackend` 纯 Python 2D A*

当前运行状态(S100P, `lingtu.py nav`)

```
[INFO] PCT ele_planner loaded: /home/sunrise/data/nova/maps/active/tomogram.pickle
       map_dim=[95, 71]  slices=2
[INFO] PCT: extracted 2D grid for goal safety BFS: shape=(71, 95) res=0.200
```

- `map_dim=[95, 71]`:95×71 网格,分辨率 0.2m,约 19m × 14m 场地
- `slices=2`:当前地图两层(地面 + 一层高物)
- goal safety BFS: `global_planner_service.py:_find_safe_goal()` 把 goal 挪到最近可通行 cell,避免用户点在墙里

平台约束

平台	PCT 可用	实际 backend
S100P (aarch64)	✅ .so 原生编译	`pct`
Windows / x86 开发机	❌ binary 不兼容	自动 fallback `astar` (2D 纯 Python)
CI 容器	❌	`astar`

---

2. Tomogram — PCT 的输入

什么是 tomogram

Tomogram 是把点云"切片"成**多层可通行性图**的数据结构: - 每层 = 一个高度区间(地面 / 1 米高 / 2 米高) - 每格 = 该层在该 XY 位置的(可通行性,elevation,gradient) - 层间关系 = "从第 N 层到第 N+1 层需要跨越多少高度" → 建模台阶/斜坡

构建时机

地图建图完成后,REPL / HTTP 调用 `map save <name>` 触发: 1. `/nav/save_map` → Fast-LIO2 写 `map.pcd` 2. `/pgo/save_maps` → Pose Graph Optimization 写 `poses.txt` + patches 3. `_build_tomogram(pcd, elevation_map)` → **`tomogram.pickle`** ← PCT 读这个 4. `_build_occupancy_snapshot` → `occupancy.npz` + `map.pgm` + `map.yaml` (RVIZ 兼容)

存放: `~/data/inovxio/data/maps/<map_name>/tomogram.pickle`

建图后没 build tomogram 会怎样

`session_start navigating {map_name}` 会检查 `tomogram.pickle` 存在:

```
if not os.path.isfile(os.path.join(base, "tomogram.pickle")):
```

```
    return error("Map has no tomogram — build it first")
```

强制要求,否则 PCT 没输入跑不了。

---

3. 全局规划 — `_PCTBackend.plan()` 展开

调用位置: `NavigationModule._plan()` 收到 goal_pose 后。

步骤

```
# 入口:src/global_planning/pct_adapters/src/global_planner_module.py
backend.plan(start=np.array([x, y, z]), goal=np.array([x, y, z]))

# TomogramPlanner.plan() 内部(planner_wrapper.py):
1. world → grid index   (pos2idx)     # (-1.5, 2.3) → (i=7, j=19)
2. 高度 → slice index   (pos2slice)   # z=0.3 → layer 0, z=1.1 → layer 1
3. C++ ele_planner.so 做 3D A*:
   - Hex-grid 邻接(6 邻)+ 垂直邻接(上下 slice)
   - cost = 距离 + traversability 权重 + slice 切换惩罚
   - 结果:grid index 序列 [(i0,j0,s0), (i1,j1,s1), ...]
4. C++ traj_opt.so 做 GPMP 轨迹优化:
   - Gaussian Process Motion Planning
   - 最小化 jerk + 保持避障 + 端点约束
   - 输出平滑连续曲线
5. grid index → world  (transTrajGrid2Map)
   - 输出:np.ndarray (N, 3)  [[x, y, z], ...]
```

输出

- 若 success: `np.ndarray (N, 3)` ,N 通常 50-200 个点
- 若 fail: `[]` (空 list)→ NavigationModule mission_status=FAILED,外场 toast"规划失败"

性能(S100P,30m 场地)

阶段	耗时
pos2idx / pos2slice	<1 ms
ele_planner 3D A*	30-150 ms
traj_opt GPMP	40-120 ms
grid→world	<1 ms
总计	**80-300 ms**

---

4. A* 备胎 — `_AStarBackend`

跨平台兼容(x86 / CI)用。

- 纯 Python `heapq` + 8 连通
- 只用 tomogram 的 **ground-floor slice**(单层 2D)
- 无轨迹优化,输出格栅锯齿路径
- 200×200 格子数秒量级,性能差

仅用于:开发测试,不用于生产导航。详见 `docs/02-architecture/PLANNER_SELECTION.md`。

---

5. 全局 → 局部 解耦:WaypointTracker

PCT 全局路径是**稀疏长路径**(30m 可能 200 点),直接跟踪一帧一跳不合理。中间层 `WaypointTracker` 负责拆解:

职责

- 存:完整 global_path + 当前 `current_waypoint_index`
- 推进:每帧 `advance_if_reached(robot_pos)`,到达半径 `waypoint_threshold=0.5m` 就 `index += 1`
- 卡住: `is_stuck(robot_pos)` — 2s 内位移 <0.1m 判定,触发 recovery
- 下采样:global_path 上点太密时按 `downsample_dist=0.8m` 间隔取

NavigationModule 定频广播

`navigation_module.py:635` 每帧(10 Hz):

```
self.waypoint.publish(Pose(
    position=current_waypoint_world,
    orientation=look_at_next_waypoint_yaw
))
```

语义:“**局部规划器,你现在朝这个点走**”。LocalPlanner 不关心后续 waypoints,只处理当前这个。

---

6. 局部规划 — CMU 预采样路径

源:CMU `base_autonomy` 开源(TARE 原班人马 Chao Cao)。 我们的包装: `src/base_autonomy/modules/local_planner_module.py`。 C++ 核心: `src/nav/core/include/nav_core/local_planner_core.hpp`。

算法思路

不走在线优化(MPC),而是**离线生成候选路径库,在线快速打分**。

候选路径库(离线,MATLAB 生成)

`src/base_autonomy/local_planner/paths/`:

```
pathList.txt            ~1000 条候选路径(PLY 格式)
correspondences.txt     每条 path 与 voxel 的对应关系
pathAll.ply             可视化
startPaths.ply / paths.ply
```

结构: - 7 组(group),每组 ~150 条路径 - 每条 path 是一条**长度 1-2m 的弧线**,固定 3D 形状 - 组与组按方向分布(直走 / 左偏 / 左弯 / ...)

在线打分(每帧 10 Hz)

输入: - `waypoint` — 来自 NavigationModule,当前目标点 - `terrain_map` — 来自 TerrainModule,障碍物点云(已滤地面) - `odometry` — 机器人当前位姿 - `boundary` — 来自 GeofenceManagerModule,硬边界 - `added_obstacles` — 外部注入的障碍

流程(`local_planner_core.hpp`):

```
1. 按 robot yaw 把候选路径库旋转到 body frame (36 个方向,OpenMP 并行)
2. 对每条候选 path,检查 voxel 是否和 terrain_map 中障碍重合 → 标记 blocked
3. 对每条可通行 path,scorePath():
     score = (1 − √√(方向误差权重)) × 旋转代价² × terrain可通行权重
4. 按 group 聚合,选最优组,从该组内选最优单条 path
5. 发布为 nav_msgs/Path(20-30 点)
```

加速

技术	收益
SoA (Structure of Arrays) + CSR 稀疏	cache 连续,消除 stride-4 访存
`scorePathFast` LUT(替代 sqrt(sqrt))	**2.08x** 速度
OpenMP 并行 36 方向	aarch64 四核线性加速
xsimd ARM NEON / x86 AVX	批量旋转点云
LTO + `-ffast-math`	跨函数内联

输出

`nav_msgs/Path`,20-30 点,约 1.5-2m 前瞻,机器人坐标系下的弧线。

---

7. 路径跟踪 — PathFollower (Pure Pursuit)

源:CMU `base_autonomy` 同源。 我们的包装: `src/base_autonomy/modules/path_follower_module.py` 。 C++ 核心: `src/nav/core/include/nav_core/path_follower_core.hpp` 。

Pure Pursuit 核心

给定: - robot pose (x, y, yaw) - local_path (沿途点序列) - current velocity v

算法:

```
1. 自适应 lookahead:
     L = clamp(v × k_lookahead, L_min, L_max)
     = clamp(v × 0.4, 0.5, 2.0)    默认参数
2. 在 local_path 上找距离机器人 L 的前瞻点 P
3. 令 α = heading_to_P − robot_yaw
4. 曲率 κ = 2 sin(α) / L
5. cmd_vel.linear.x  = v_target (配置项,默认 0.6 m/s)
6. cmd_vel.angular.z = κ × v_target
7. 加速度限幅:avoid v_target 跃变 → 平滑上升
```

四足适配

- v_target 比较保守(0.3-0.8 m/s),四足抖动容忍度低
- `no_rot_at_goal=true` :到终点不原地转,避免抖
- `no_rot_at_stop=false` :急停后允许原地转绕障

---

8. CmdVelMux — 优先级仲裁

所有产生 cmd_vel 的模块都**不直接**发给 driver,而是统一经过 `CmdVelMux` :

源	优先级	超时
TeleopModule(摇杆/WS)	**100**	0.5 s
VisualServoModule(视觉伺服)	80	0.5 s
NavigationModule recovery(倒车)	60	0.5 s
PathFollowerModule(正常巡航)	40	0.5 s

仲裁规则: - 每个源独立发到自己的 In port - 最高优先级 “active”(最近 0.5s 内有发)的源胜出 - 无源 active 时输出零速(安全默认)

意义:操作员摇杆一动,PathFollower 自动让位;视觉伺服追丢目标 0.5s 后,自动回退到 PathFollower。

---

9. NavigationModule 状态机

`src/nav/navigation_module.py` 的 mission FSM:

```
IDLE ── goto ──→ PLANNING ── ok ──→ EXECUTING ── arrived ──→ ARRIVED
  ↑                 │                  │                        │
  │                 fail               │ stuck                  │
  └── cancel ─────── FAILED ←──────── RECOVERING ─── can't recover ┘
```

状态含义与暴露

外场通过 `lt status` (或浏览器 Topbar)看 `mission.state`:

状态	含义
IDLE	空闲,可接受 goal
PLANNING	正在调 PCT
EXECUTING	跟 global_path 中
RECOVERING	卡住,正在倒车 + replan
ARRIVED	到达目标半径 0.5m 内
FAILED	规划失败 / 超 max_recoveries / 目标不可达
CANCELLED	用户 `lt stop`

Recovery 路径

卡住(2s 位移 <0.1m)→ 触发: 1. 广播 `recovery_cmd_vel` (后退 0.3s @ 0.3 m/s) 2. 重新调 `_plan()` 再规划 3. 连续 3 次卡住 → FAILED

---

10. 和语义层的关系

这套”**几何规划栈**”(PCT + Local + PathFollower)下游,还有一层**语义**负责把自然语言 / 视觉目标转成 goal_pose:

```
用户说"去厨房找红色椅子"
   ↓
SemanticPlannerModule
   ├─ GoalResolver (Fast-Slow) — 把"厨房"解成坐标
   ├─ VisualServoModule       — "找红色椅子"切视觉伺服
   └─ AgentLoop               — 多步 agent(LLM tool calling)
   ↓
goal_pose 或 cmd_vel(视觉伺服走旁路)
```

```
    ↓
上述几何规划栈
```

见 `docs/06-semantic-nav/` 目录。几何层对语义层透明:**收到 goal 就规划,不关心 goal 从哪来。**

---

11. 关键文件快查

文件	作用
`src/global_planning/PCT_planner/planner/scripts/planner_wrapper.py`	TomogramPlanner
`src/global_planning/PCT_planner/planner/lib/ele_planner.so`	C++ 3D A*
`src/global_planning/PCT_planner/planner/lib/traj_opt.so`	GPMP 轨迹优化
`src/global_planning/pct_adapters/src/global_planner_module.py`	`_PCTBackend` / `_AStarBackend` 注册
`src/nav/global_planner_service.py`	统一 plan() 入口 + goal safety BFS
`src/nav/navigation_module.py`	Mission FSM + WaypointTracker 调度
`src/nav/waypoint_tracker.py`	到达检测 + 卡住检测
`src/base_autonomy/modules/local_planner_module.py`	CMU LocalPlanner Python 包装
`src/nav/core/include/nav_core/local_planner_core.hpp`	CMU LocalPlanner C++ 核心
`src/base_autonomy/modules/path_follower_module.py`	Pure Pursuit Python 包装
`src/nav/core/include/nav_core/path_follower_core.hpp`	Pure Pursuit C++ 核心
`src/nav/cmd_vel_mux_module.py`	优先级仲裁
`config/robot_config.yaml`	lookahead / v_target / waypoint_threshold 等所有参数

---

12. 已知问题 / 技术债

#	问题	影响	状态
1	PCT 只有 aarch64 binary	开发机 / CI 不能跑生产 planner	接受(用 A* 备胎),无 ROI 重编译
2	tomogram 离线构建	exploring 模式(未知场地)不能用 PCT	接受,explore 用 frontier + A*
3	PCT 失败处理简单	start/goal 在障碍里 → BFS 挪到最近 cell,不保证可解	已有 LERa 恢复路径(3 步 look-explain-replan)
4	GPMP 不处理动态障碍	规划时 tomogram 是静态快照	局部层 LocalPlanner 兜底(terrain_map 实时更新)
5	LocalPlanner paths/ 是 MATLAB 生成	想改路径分布要重跑 MATLAB	低优,当前 1000 条够用
6	CmdVelMux 超时固定 0.5s	某些场景(语义规划慢)可能断连	可配置但少用

---

13. 性能汇总(S100P aarch64)

阶段	耗时
PCT 全局规划(30m 场地)	80-300 ms
LocalPlanner 一帧(1000 path × 36 rot)	3-8 ms
PathFollower 一帧	<1 ms
CmdVelMux 仲裁	<0.1 ms
端到端 goal → cmd_vel(稳态)	**<15 ms**
端到端 goal → 机器人开始动(冷启 PCT)	**~300 ms**

---

14. 想深入看哪块

- **ele_planner 3D A* 怎么对台阶打分** → 读 `src/global_planning/PCT_planner/planner/src/ele_planner/` C++ 源, `TomogramPlanner.plan()` 论文第 3 节
- **GPMP 轨迹优化** → 论文第 4 节 + `traj_opt.so` 绑定

- **CMU LocalPlanner scorePath 系数调优** → `config/robot_config.yaml:local_planner.*` + benchmark 在 `src/nav/core/test/`
- **PCT 失败的 LERa 恢复** → `src/semantic/planner/.../action_executor.py` 的 Look-Explain-Replan
- **Fast-Slow 语义规划** → `docs/06-semantic-nav/` + `goal_resolver.py`

---

15. 相关文档

- `docs/02-architecture/PLANNER_SELECTION.md` — PCT vs A* 选型决策 + 运行时链路速查
- `docs/02-architecture/ARCHITECTURE.md` — 整个 LingTu 模块/Blueprint 架构
- `docs/02-architecture/ALGORITHM_REFERENCE.md` — 各算法的学术出处
- `docs/01-getting-started/FIELD_OPERATION_ZH.md` — 外场操作手册(怎么用这套栈)
- `docs/06-semantic-nav/` — 语义规划层